Security Overview

Early Access · MessageFoundry v0.3.2 · July 2026

MessageFoundry is a healthcare interface engine that carries PHI, so security is a first-class design constraint rather than an afterthought. This overview describes the controls that are built into the engine today: how it authenticates and authorizes every action, how it protects data at rest and in transit, how it produces a tamper-evident record of what happened, the development and CI practices behind the code, and the threat model for each interface. It closes with an honest statement of our verification posture — what we hold ourselves to, and what has **not** been independently reviewed.

This page is a summary. For the full engineering detail behind these controls, see the **Secure Development Standards** document (linked at the end). Where a control is opt-in or off by default, that is called out explicitly — we would rather be precise than impressive.

Secure by default

MessageFoundry is built to fail closed and to make the safe configuration the default one.

- **Authentication is on by default, and cannot be turned off toward the network.** The running service attaches an authentication layer to every route apart from a small unauthenticated set — a `GET /health` liveness probe, the sign-in, step-up and MFA pages (a gate that demanded a fresh step-up in order to perform one would deadlock), a non-sensitive operational-config endpoint, and the console's static assets. Authentication *can* be switched off on a loopback bind, an embedding and development posture in which every request runs as a full-privilege system identity, so neither RBAC nor the field-level gate withholds anything. Starting that way on a **non-loopback** bind is refused outright at any enforcement level, and the deviation is warned at startup and named in the running posture view. The in-process embedding path is fail-closed by the same rule: with no authentication service attached it denies every protected route until a developer explicitly opts out for local development.
- **Loopback by default.** The API binds to `127.0.0.1`. On a PHI instance at the shipped enforcement level, a non-loopback *plaintext* bind is **refused at startup** — so exposing a listener beyond loopback requires TLS (in-process or terminated at a trusted upstream proxy), and bearer tokens and PHI never cross a network in the clear by accident. Overriding that refusal takes a named switch — lowering the enforcement level, or declaring that the instance carries no real patient data — and every such switch is audited at startup and surfaced in the running posture view. There is no way to do it quietly.
- **Deny-by-default authorization.** Every protected route demands a specific permission. Anything not explicitly granted is denied.
- **PHI at rest is key-required, not key-optional.** On any instance that handles real patient data — which all three built-in environments do by default — the service **refuses to start without an encryption key**. Running keyless is a loud, audited opt-out, not a quiet default. (Details under *Encryption at rest*.)

- **First-run bootstrap is sealed.** On first start against an empty store, the engine creates a single bootstrap administrator with a random one-time password written to an owner-only file (never to the log). The account is flagged to force a password change on first sign-in, and it self-retires once a real second administrator exists (or after a configurable expiry — 72 hours by default — if left unclaimed).

Identity and access control

Authentication

Operators authenticate as **local accounts** or against **Active Directory**, and every action is attributed to a distinct user in the audit trail — no shared logins.

- **Local accounts** use **argon2id** password hashing (tuned cost parameters, with a concurrency cap so a login flood cannot exhaust CPU). The password policy is aligned to OWASP ASVS 5.0: a 15-character minimum, no mandatory character-class composition (ASVS discourages it), offline breached- and common-password screening, a context-word deny-list, and rejection of passwords that contain the username. All screening is local — no third-party password API is called.
- **Active Directory accounts** (the AD pathway is off until an operator enables it) authenticate by LDAP bind over **LDAPS** (TLS 1.2+, with certificate verification and nested-group resolution). AD security groups map automatically to engine roles — and to per-channel scope — and roles re-sync from the directory on every login. With AD enabled, a background pass re-resolves signed-in directory principals and revokes the sessions of accounts the directory no longer recognises — typically within two intervals (the interval is five minutes by default, and two agreeing probes are required before anything is revoked). It is deliberately conservative, and worth understanding before you rely on it: a per-pass bind budget means a large estate degrades to a longer effective interval rather than a bind storm against the domain controller; a circuit breaker aborts a pass that would revoke a large *and* broad share of signed-in sessions, revoking nothing and raising an alert instead, because a broken search base looks identical to everyone being disabled; and it fails open when the domain controller is unreachable. Treat it as a backstop, not as a replacement for the session lifetime cap or for the offboarding itself.
- **Multi-factor authentication.** Local accounts use a native **RFC 6238 TOTP** second factor with single-use recovery codes. MFA is **required for local accounts by default**, enforced as an access gate; the documented org opt-out is a single setting, and the enrollment path is deliberately reachable so a required-but-unenrolled admin can never deadlock. The TOTP secret is stored encrypted at rest and recovery codes are argon2id-hashed. **WebAuthn/FIDO2 passkeys** are also built for the browser console (an optional install extra): the browser step-up stays two-credential — the passkey satisfies the MFA leg while the password re-proof still stamps step-up freshness, so enrolling a passkey never silently relaxes the re-proof — and a passkey is bound to the configured public origin, failing closed behind an undeclared reverse proxy. Passkeys are asserted at `user_verification=preferred`, so treat one as a possession factor alongside the password rather than as a self-contained two-factor ceremony, and keep TOTP enrolled for clients outside the browser. **AD and Kerberos MFA is the directory's responsibility, and the engine cannot verify it.** A directory sign-in is never prompted for an engine TOTP, and the engine issues the session as MFA-satisfied on the strength of the bind alone — no second-factor evidence is received from the domain controller. So if directory operators must present a second factor, that has to be enforced in the directory; the engine's own MFA gate will not do it for

them. Optional browser **OIDC federation** (off by default) is the one delegated pathway that carries engine-side evidence: by default a sign-in whose signature-verified token asserts no configured MFA claim is refused. That gate is on by default and can be disabled.

Authorization (RBAC)

Access control is **role-based and deny-by-default**, over a single catalogue of 27 permissions. Six built-in roles ship — Administrator, Operator, Deployment, Coding, Viewer, and Auditor — and **custom roles** layer on top as an additive overlay: a custom role is a named *subset* of the same catalogue, can never invent a new permission kind, and can never be granted the escalation primitives (user management, approvals, DR operation). Holding multiple roles grants the union of their permissions. Every request re-resolves the caller's roles and permissions from server-side state (no caching), so a change made *in MessageFoundry* — a revoked role, a disabled account, a revoked session — takes effect on the next request. A change made in the *directory* is a different matter: it propagates through the reconciliation pass described above, not immediately.

Two finer-grained controls layer on top of route-level RBAC:

- **Per-channel scoping** confines an operator's message and connection access to a defined set of interfaces. Out-of-scope message reads return `404` (so they never reveal that a record exists), and denials are audited.
- **Field-level (per-property) authorization** gates individual PHI-bearing fields *within* a response, so a user can see an object without seeing its patient-identifying fields. The policy lives in one place and is applied to every returned row through a single helper, and a CI guard compares that policy against the documented map in both directions — so a new PHI-bearing field cannot be added without being mapped, and the map cannot invent a field that does not exist. Patient summaries (MRN, name) and free-text disposition fields require a summary-view permission; the raw message body requires a separate, coarser permission.

Defense-in-depth for sensitive actions

High-value and administrative operations are protected by several independent layers, not by network location alone:

- **Step-up re-verification** — sensitive admin operations require a recent credential re-proof (having satisfied MFA), on a sudo-style freshness window: five minutes by default, seeded by sign-in and re-armed by an explicit re-proof. Bulk PHI reads (search, export, layered search) sit behind the same gate. Because sign-in seeds the window, a session that has just authenticated already satisfies it — which is why the routes that bind a *new* authenticator (TOTP enroll and confirm, disabling MFA) additionally demand a fresh, single-use proof bound to that specific action, something a session hijacked inside the window cannot supply. That action-bound proof is **on by default**; the documented org opt-out is a single setting that turns it off and reverts those routes to the ordinary session-wide step-up window.
- **Dual-control approval** — high-impact operations (bulk dead-letter replay and connection purge) can be configured to require a second, distinct approver before they execute. It is **opt-in and off by default**, so a single-operator site is never blocked; a requester can never approve their own request, and both identities are written to the audit log.

- **Contextual-risk step-up** — an optional signal (off by default) forces a fresh step-up when a sensitive admin action arrives from a client address the session has not verified from. It is advisory and step-up-forcing only — it never changes an allow/deny decision.
- **Brute-force and abuse limits** — a per-account lockout on local accounts, a sign-in sliding window applied **per client IP and globally**, per-actor limits on credential ceremonies, and a per-actor budget on bulk PHI reads so a scripted client cannot outrun the same throttle that bounds an operator. Note the split: the per-account lockout covers local accounts only. AD, Kerberos and OIDC sign-ins are covered by the sliding window plus the domain's or IdP's own lockout policy, so a deployment that turns that window off is relying entirely on the directory for anti-automation on exactly the pathway most sites use.

Interface authentication

Beyond operator sign-in, the engine authenticates its **machine interfaces**:

- **mTLS** — mutual-TLS client-certificate authentication on transports that support it, including a non-interactive service-identity route that accepts *no* bearer token at all: a certificate chaining to the pinned client CA plus a deny-by-default allow-list of qualified certificate names. It is inactive until both a client CA and an identity map are configured. Admission is by chain-and-name, and **revocation is by allow-list removal rather than OCSP or CRL** — strict path validation is not revocation checking — so the allow-list is the revocation surface and should be operated as one, with live revocation remaining your PKI's job. That plane is fenced away from PHI permissions by construction.
- **OAuth 2.0 client-credentials** — for REST/FHIR outbound calls.
- **SMART on FHIR Backend Services** — OAuth 2.0 `client_credentials` with a signed-JWT client assertion (RS384/ES384), opted in per connection; the engine mints a per-request bearer token and re-mints on a `401`, and the token endpoint is governed by the outbound egress allowlist. This is a client-only profile — there is no authorization-server facade.

Sessions

Sessions are **opaque server-side tokens** (not JWTs): the client holds the token, and the store keeps only its SHA-256 hash, so logout, expiry, and role changes take effect immediately. Each request enforces an **idle timeout** (default 30 minutes) and an **absolute lifetime** (default 12 hours) — the pair aligns with NIST SP 800-63B AAL2 reauthentication guidance. Session validation **fails closed on a backward wall-clock step** (for example an NTP step-back or a VM snapshot revert) rather than reviving an expired token, and the idle clock advances only on user-driven requests.

Changing a password, disabling a user, or an AD role/scope change on re-login revokes that user's sessions. A per-user concurrent-session cap applies (default 5). Users can list and revoke their own active sessions (including "sign out everywhere else"), and administrators can force-sign-out a user for offboarding or suspected compromise. Every revocation is audited. The browser console holds its session in an **HttpOnly,** `SameSite=Strict` cookie that JavaScript cannot read, and every state-changing console request is additionally origin-checked server-side, so a browser that ignores `SameSite` still cannot be driven cross-site. Its path is the site root rather than `/ui` — deliberately, so a same-origin WebSocket handshake at the root can carry it — which means the confinement is enforced by the server rather than by the path: only the console's own `/ui` routes ever read the

cookie, and the JSON API authenticates from the `Bearer` header alone and never from it. Over HTTPS the cookie additionally carries `Secure` and the browser-enforced `__Host-` prefix, which binds it to that exact host. Non-browser clients (the test harness, automation) send the token as a `Bearer` header, keep it in memory only, and refuse to send credentials over plaintext HTTP to a non-loopback host.

Data protection

Encryption at rest

The PHI-bearing columns of the message store are encrypted with **AES-256-GCM** (authenticated encryption): raw inbound bodies, the queued copies that carry a message through the pipeline, transformed outbound payloads, detached attachment chunks, captured partner replies, patient summaries, handler metadata, and the free-text error/disposition columns. Ciphertext is **bound to the cell it lives in** by default, so a value cut and pasted from one row or column into another fails its authentication tag rather than being silently accepted. Keys are managed as a fingerprint-based **keyring** (an active key plus retired decrypt-only keys) with an offline key-rotation command. A row that cannot be decrypted is routed to dead letters rather than crashing the service.

Key material never lives in the config file. It is supplied from the environment, or on Windows from a machine-bound DPAPI-protected key file, or through a pluggable key-provider seam that can envelope-decrypt a wrapped key inside an external HSM/KMS/Vault module; a Vault/OpenBao Transit mode goes further and performs every encrypt and decrypt inside Vault, so the data key never enters engine memory. Selecting an unavailable provider fails closed — the service refuses to start rather than silently falling back to plaintext.

Three honest deployment notes apply. First, **at-rest encryption is not optional on a PHI instance**: with no key configured, the service refuses to start, and running keyless anyway takes a named, audited opt-out that still emits an unencrypted-at-rest warning — and under the strictest enforcement posture, a second, separate acknowledgement on top of it. Second, an application-level cipher cannot reach the database *substrate* — write-ahead and transaction logs, temp/version stores, and indexes — and a set of columns is deliberately left plaintext because it has to be looked up or indexed. The routing keys (message type and control ID) are the low-sensitivity members of that set, but it also includes the lookup **keys** of transform state and reference snapshots, which are PHI-capable if a handler keys them on a raw identifier instead of a surrogate. We would rather point you at the list than summarize it and get it wrong: `docs/PHI.md` §2 is the normative, per-column inventory and names every plaintext column with its sensitivity. Closing that gap is the role of **operator-supplied whole-database or full-volume encryption** (SQLCipher or BitLocker/LUKS for the embedded store; TDE plus volume encryption on the database host for SQL Server or PostgreSQL). The engine documents this as a deployment precondition; it cannot verify or enforce it, and we do not claim otherwise.

Third — and the store is not the whole at-rest story. PHI also lands on disk *outside* the column cipher, and the largest such surface is the **File connector's own directories**: a file-based feed writes and spools full clinical bodies (`.hl7` , `.processed` , `.error`) as plaintext, with no engine cipher on that path at all. The backup pipeline stages a snapshot and, when post-backup verification is enabled, a decrypted archive through the operating system's temp directory rather than the hardened data directory; the optional uploaded-logs directory is created owner-only on a best-effort basis and gets no engine-applied ACL on Windows; and application log files are

plaintext. For a hospital running file-drop feeds this is not an edge case, so we would rather state it than let the encryption section imply the store covers everything: these locations are protected by volume encryption, directory ACLs, and pointing the temp path at an owner-only volume — not by the cipher described above.

Redaction and logging

PHI is kept out of logs and exception text by construction. An exception-path redaction chokepoint scrubs HL7-shaped content from any value on its way to a log or a stored error column while preserving the exception type, and a global filter chain runs on **every record emitted by the engine process** — redacting PHI (including chained exception tracebacks), scrubbing the values of credential-bearing query parameters, and stripping control characters to defend against log injection. Credentials, tokens, and message bodies are never logged. A **production** instance is **refused at startup** if it is configured for debug-level logging. Two limits on that refusal, both documented, and both worth knowing before you lean on it. It keys on the **production tier**, so a non-production instance that nonetheless carries real patient data — a PHI staging box, typically — can still be started at DEBUG. And it is a **startup** gate only: an operator holding the diagnostics permission can raise the *live* level to DEBUG on a running production instance, with no posture check. That change is audited with the actor and the old and new level, and a restart re-asserts the configured level (a config reload does not), but nothing refuses it. Both are operator decisions, and both are ones to make deliberately, because the application log is itself a PHI read surface (below).

Stated honestly: redaction is conservative, not de-identification. It rewrites HL7-shaped spans, date runs, and multi-token name runs — an adversarially crafted *single-token* identifier can still survive it, which is why the application log is itself classified as a PHI read surface and gated behind a permission.

Retention

A retention runner blanks PHI **in place** on a configurable window — body, patient summary, and metadata — while keeping the message row, so counts, disposition, and audit history survive the purge. Every pass that does real work writes a single audit entry with cutoffs and counts (never content).

Two of these windows are genuinely enforced, and the rest are not — the distinction matters for a data-minimisation review, so here it is plainly. **Enforced:** the inbound PHI-body window and the dead-letter body window. Under the default enforcement posture the service **refuses to start** with either of them unbounded, and a non-enforcing PHI instance auto-bounds them to 30 days; keeping PHI indefinitely is an explicit, audited opt-out. **Available but unset:** transform state (which by design may hold a correlation map), saved search filters, alert and connection-event detail, and application log files each ship with their own window *available*, defaulting to keep-forever, and nothing refuses to start if you leave them that way — bounding them is a deployment step. A small number of tiers have no purge path at all yet, including a legacy outbound-body table on stores upgraded from an older SQL Server schema. So do not read this section as a promise that all PHI ages out of the system on its own. Audit-log pruning, separately, is deliberately *not* performed — the trail is keep-forever by design.

Tamper-evident audit

Every authentication event, every permission **denial**, every PHI access, and every state-changing authorization is written to a durable audit log with the acting user — sign-ins, failures, lockouts, permission denials, user and role changes, AD mapping changes, and PHI access (viewing a raw message or displaying patient summaries is recorded with the viewer). Authorization *grants* on ordinary read and monitoring routes are **not** recorded by default, because console polling would flood the chain; full authorization tracing including reads is a single setting, off by default. The ePHI-access audit floor is unconditional and cannot be switched off. Each row also records the caller's client address, so the trail answers *where from* as well as *who*. Credentials, tokens, and PHI bodies are never written to the audit trail.

Each audit row carries a `row_hash` chaining the previous row's hash with this row's content, and the client address is folded inside the chain, so attribution cannot be rewritten without breaking it. The chain is verified with a command-line tool (`messagefoundry audit-verify`). Two properties of it are worth stating precisely, because a reviewer who reads the verifier will find them anyway.

Keyed or keyless, depending on the store key. With a store encryption key configured the chain is **keyed** — an HMAC under a key derived from that key, or, under the Vault/OpenBao Transit cipher, a MAC computed inside Vault so no key material ever enters engine memory — and it is keyed on all three store backends, SQLite, SQL Server and PostgreSQL alike. An attacker who can write audit rows but does not hold the key cannot forge a self-consistent chain. On a **keyless** store — no key configured, so at-rest encryption is off — the chain is a plain SHA-256 chain instead: tamper-evident against accidental or careless modification, but not forgery-resistant against someone who can already write to the table. That is a reason to set the store key beyond confidentiality alone. An existing keyless chain is upgraded only by an explicit, non-silent re-key step that re-verifies the old chain before keying it — never quietly at startup.

The walk catches insertion, editing and reordering — not tail truncation. Deleting the *newest* rows leaves a shorter chain that still verifies cleanly, so the walk alone will not see it. Catching that needs the chain's **anchor** — its row count plus head hash — captured out-of-band and compared back; the store exposes that anchor and the disaster-recovery restore path reconciles against it today, but the shipped verify command does not yet accept one. Until it does, the practical control against a deleted tail is the off-box copy described next.

Either way this is **tamper-evident, not tamper-proof**: it *detects* on-host modification rather than preventing it, so the store's file permissions should be locked down and the service run with least privilege. Because the hash chain lives on the same host as the data it protects, both the general log **and the audit rows themselves** can be forwarded off-box to a syslog or SIEM collector so an independent copy survives a host compromise. The same redaction filters apply to the forwarded stream, and the hop can be encrypted natively (RFC 5425 TLS-syslog, verifying the collector against an explicit trust anchor) without needing a local forwarding agent.

Transport security

Native transport TLS ships for both the API/WebSocket plane and the MLLP data plane, with a reverse-proxy hardening path for TLS-terminating upstreams:

- **API / WebSocket TLS** — in-process TLS with a TLS 1.2+ floor and configurable certificates, cipher suites, and minimum version, plus an optional client CA for mTLS. An operator cipher string is validated at config load and rejected if it would admit a non-forward-secret suite.
- **MLLP-over-TLS** — per connection, with a required server certificate, peer verification by default, and optional mutual TLS.
- **Outbound TLS verification fails closed** for LDAPS, database, and REST egress; it can be overridden only through an explicit insecure-TLS escape that logs a loud warning. Database connections are TLS-verified by default, and a private internal CA can be trusted by pinning it rather than by disabling validation.

TLS is **off by default** because the default posture is loopback-only. As noted above, a non-loopback *plaintext* bind is refused at startup on a PHI instance at the shipped enforcement level — so any deployment that exposes a listener beyond the local host must supply TLS, in-process or at a trusted upstream proxy, unless an operator has deliberately lowered enforcement or declared the instance free of real patient data. Both of those are audited and visible in the posture view.

Application and input security

All inbound HL7, file, and configuration content is treated as untrusted **data, never instructions**, and no message is ever silently dropped — every received message is persisted with a disposition, and failures land as error or dead-letter dispositions.

- **Parsing** is two-tier: a tolerant parser on the hot path with opt-in strict validation, and size and segment caps enforced *before* parse (16 MiB / 10,000 segments).
- **Injection defenses are uniform** — parameterized SQL throughout, an ODBC connection-string guard, and RFC 4515 LDAP filter escaping.
- **Request and file limits** — an HTTP body cap with chunked-body rejection, a file-input cap with filename sanitization and path-traversal defense.
- **SSRF hardening** — REST destinations refuse redirects, and webhook alert sinks enforce an `http(s)` scheme with an optional host allowlist and a no-redirect opener.
- **DoS caps** — bounded MLLP frame size, connection count, and idle timeout; bounded file, HTTP, and WebSocket limits. The WebSocket Origin is checked against an allowlist before the connection is accepted, and interactive API docs are disabled by default.
- **Browser-surface hardening** — the operator console escapes all message content under a strict content-security policy, serves PHI reads `no-store` so nothing is cached to disk, and neutralizes browser-active attachment types at serve time instead of rewriting the stored bytes. **Nothing PHI-bearing is written to browser storage** — the only stored item is the session cookie, and `localStorage` holds table column widths and sort order, nothing else. Ordinary console URLs carry opaque ids, and every page is served — and repeats in-document — `Referrer-Policy: no-referrer`. **One exception you should hear from us rather than find:** message content search is a `GET` form, so its needle (`?content=…` , `?field_value=…`) sits in the URL, and a PHI-shaped search term can therefore reach browser history and the engine's own request log, where redaction does not catch a single-token identifier. What offsets it today: the search page is step-up-gated, the needle is

deliberately dropped rather than replayed across a step-up redirect, `no-referrer` keeps it out of `Referer`, and a saved or layered search holds the needle server-side in an encrypted column so only preset ids travel in the URL. Getting the live needle out of the URL is an open item, tracked in `docs/PHI.md`.

- **Safe error responses** — clients receive generic `500` responses with no stack traces.

Outbound destinations are **deny-by-default on a PHI instance**. Every transport that can carry PHI off the box — MLLP, TCP, file, HTTP/REST, database, SMTP, and Direct — is governed by a fail-closed allowlist enforced at config load, reload, and start, and a PHI instance left with fully-open egress **refuses to start** at the shipped enforcement level. Turning deny-by-default off is a named, audited loosening, surfaced in the posture view.

Secure development lifecycle and CI security gates

Engineering controls are a relative strength of the project, and the core set runs as **blocking** gates in continuous integration — a violation fails the build:

- **Static analysis (SAST)** — Bandit plus custom Semgrep rules that forbid known-dangerous patterns (shell execution, `eval` / `exec`, insecure deserialization, disabled TLS verification).
- **Dependency auditing (SCA)** — `pip-audit` over a **hash-pinned lockfile** (so the audit is reproducible), an equivalent audit of the VS Code extension's npm tree, a daily scheduled run, a lockfile drift guard, and Dependabot updates.
- **Secret scanning** — full-history scanning in CI, mirrored as a pre-commit check.
- **A customer/PHI leak guard** and a cryptographic-inventory discovery gate, both of which also fail the build.

Advisory alongside them — running on every change, reported to reviewers, but **not** merge-blocking: **CodeQL** code scanning on the extended security query suite (on every push and pull request plus a weekly schedule), container-image scanning, and supply-chain posture scoring. CodeQL is advisory for a specific and, we think, creditable reason: uploading its results needs a permission that pull requests from forks do not carry, so requiring it would block every outside contribution. We would rather say which checks stop a merge than imply that all of them do.

Behind the pipeline sit a per-interface threat model, a release gate, a root-cause review of every significant vulnerability, and published remediation targets on two schedules — 7 / 30 / 90 days (Critical / High / Medium) for vulnerabilities in MessageFoundry's own code, and a separate reachability-scored schedule for dependency CVEs at 14 / 30 / 60–90 days. Those posture documents are maintained privately and made available to adopters and reviewers under NDA — the rule that decides what is published and what is withheld is itself public (see *Security documentation policy* at the end). Security-critical design decisions are governed by Architecture Decision Records, and a completed remediation ledger is published with its findings named and its method described.

Releases are tagged and **Sigstore-signed**, ship a per-release CycloneDX **SBOM** and an **OpenVEX** exploitability assessment, and use PyPI Trusted Publishing (PEP 740 attestations) with GitHub-native SLSA build provenance — so the provenance of an installed artifact can be verified, not just its contents. A private vulnerability-disclosure channel with the published targets above is in place.

Honest caveats: development is currently single-maintainer, with blocking CI gates and adversarial code review compensating for the absence of a second human reviewer; and while hash-pinned installation is documented and used in our own CI, we cannot enforce it inside an adopter's environment.

Threat model by interface

The project maintains a STRIDE-lite threat model with explicit trust boundaries. The trust boundary is the **organization's private network plus the host's OS accounts**: MessageFoundry is deployed inside a single healthcare organization's network, never internet-facing, and the model assumes a trusted operating system with correct file permissions and operator-supplied volume encryption. The engine is **single-tenant** and makes no cross-tenant isolation guarantees. The table below summarizes the posture per interface.

Interface	Primary threats	Controls
Operator API / console	Credential theft, privilege escalation, session hijack	Authentication on by default (never disableable toward the network), deny-by-default RBAC, field-level PHI authorization, MFA + step-up on sensitive actions, opaque revocable sessions, loopback-by-default with TLS required off-loopback
Inbound HL7 / MLLP	Malformed-message DoS, parser abuse, untrusted-network exposure	Pre-parse size/segment caps, frame/connection/idle caps, MLLP-over-TLS with peer verification, no-silent-drop persistence
Inbound file	Path traversal, oversized input	Filename sanitization, path-traversal defense, input size cap
Inbound X12 / DICOM	Untrusted-payload injection, content confusion	Payload-agnostic ingress (formats are not force-applied), tolerant codecs, allowlisted DICOM calling AE titles and peer IPs, the same parameterization and persistence guarantees
Outbound REST / FHIR	SSRF, credential leakage, sending PHI to the wrong place	Redirect refusal, fail-closed egress allowlist, OAuth 2.0 client-credentials / SMART Backend Services, fail-closed TLS verification
Active Directory (LDAPS)	Credential interception, filter injection	LDAPS with certificate verification, RFC 4515 filter escaping, engine-side sign-in window; lockout and MFA are the directory's to enforce and the engine does not verify them
Database egress	Connection-string injection, MITM	Parameterized queries, ODBC injection guard, fail-closed TLS verification
Audit / log forwarding	On-host tampering, PHI leakage in transit	Hash-chained tamper-evidence (HMAC-keyed once a store key is set; unkeyed SHA-256 on a keyless store), off-box forwarding for an independent copy that also covers a deleted tail, native TLS transport, redaction applied to the forwarded stream

An **inbound web-service listener** — a partner calling *into* MessageFoundry over HTTP — is deliberately **not built today**; it is a distinct surface that will need its own authentication and TLS design before it ships.

Verification posture — self-assessed, not certified

MessageFoundry is verified internally against **OWASP ASVS 5.0, targeting Level 3** — chosen above the usual Level 2 norm because the engine carries PHI. The assessment is pinned to a specific ASVS version and is re-scored when the posture changes. One thing to know up front, rather than discover: the engine's default enforcement posture changed recently, and the per-requirement re-score and sign-off against that change are **outstanding** — so the current internal scorecard trails the posture this page describes. We would rather tell you the artefact is mid-cycle than describe it as settled.

Read this part carefully. That work is a point-in-time, code-backed gap analysis conducted **by the project, on itself**. It is **not** a certification, an accreditation, a passed audit, or a third-party penetration test, and it should not be read as one. **No independent security assessment, code review, or DAST engagement has been performed.** ASVS at Level 3 *recommends* — but does not require — an independent review; ours is scheduled rather than done. A dated risk acceptance covers that gap while the project is pre-1.0 and loopback-only, and an **independent engagement is a documented prerequisite before any off-loopback or production PHI exposure**. We publish no numeric score here, because a score without an assessor's name behind it invites exactly the misreading we are trying to avoid; the current assessment and the risk register are available to adopters, evaluators, and security reviewers under NDA through the private contact route.

MessageFoundry **supports a HIPAA-compliant deployment** and maps its technical controls to the HIPAA Security Rule technical safeguards and to relevant NIST guidance. That is not a compliance claim: a HIPAA-compliant deployment also depends on administrative and physical safeguards, and on disciplined operation (volume encryption, file permissions, key management, retention, and egress allowlisting) that are the deployer's responsibility. Compliance is a property of a covered entity's whole program, assessed by that entity and its counsel. We describe what the engine does and what it does not do, and we publish our own residual risks rather than obscure them.

Deployment responsibilities

The engine cannot enforce host-level controls, so the following remain the deployer's responsibility: whole-database or full-volume encryption for the store, its journals and temp files, and file-connector directories — on a server database, that cover lives on the *database* host, not the engine host; host operating-system permissions and physical security; the database tier's own backup lifecycle; and an operational incident-response and breach-notification program.

What the engine *does* provide on this front: **encrypted, retention-bounded DR archives** of the embedded store plus its configuration bundle (on a server-database deployment the data-tier backup is handed to the DBA and the engine archives configuration only), operator DR activate/release controls, and **active-passive high availability** — one leader, warm standbys, self-fencing leases — on PostgreSQL and SQL Server. An incident-response *workflow* is not part of the product.

Learn more

- **Secure Development Standards** — the full engineering detail behind the controls summarized here — docs/Secure_Development_Standards.md

- **Security model, roles, and sessions** — docs/SECURITY.md
- **PHI handling and at-rest encryption** — docs/PHI.md
- **Security documentation policy** — what we publish, what we withhold, and how to request the rest — docs/SECURITY-DOCS-POLICY.md
- **Supply-chain transparency** — SBOM, VEX, signing, and how to verify a release — docs/SUPPLY-CHAIN.md
- **Vulnerability disclosure** — SECURITY.md

MessageFoundry is independent and unaffiliated with the vendors of other interface engines; their product names are trademarks of their respective owners. Any comparisons are factual and made in good faith.
